

Introduction to RISC-V ISA (RV32I)

What is an ISA?

- ISA (Instruction Set Architecture) is a **contract** between software and hardware
- Defines:
 - Instructions
 - Registers
 - Data sizes
 - Addressing behavior
- Does **not** define:
 - Pipeline
 - Cache structure
 - Clock speed

- We design a **microarchitecture** that implements the RISC-V ISA
- ISA specifies **what** the processor must do
- Microarchitecture specifies **how** it is done
- Same ISA can have:
 - Single-cycle designs
 - Pipelined designs
 - Out-of-order designs

RV32I Programmer's Model

- RISC-V has **32 general-purpose registers**
- In RV32I:
 - Each register is 32 bits wide
 - Program Counter (PC) is 32 bits
- PC holds the address of the current instruction
- Memory is **byte-addressable**

Program Counter Behavior

- In base RV32I:
 - Instructions are 32 bits (4 bytes)
 - Sequential execution updates:

$$PC_{next} = PC + 4$$

- Control-flow instructions modify PC:
 - Branches
 - Jumps

Load-Store Architecture

- RISC-V follows a **load-store architecture**
- Arithmetic and logical operations:
 - Operate only on registers
- Memory is accessed only via:
 - Load instructions (lw)
 - Store instructions (sw)
- No direct memory-to-ALU operations

Memory Addressing

- Memory is byte-addressable
- Each address points to 1 byte (8 bits)
- Example:
 - Address 0x00 → Byte 0
 - Address 0x01 → Byte 1
- Instruction addresses in RV32I are 4-byte aligned

Base RV32I Instructions

- RV32I defines a small set of base integer instructions
- Examples:
 - add, sub
 - lw, sw
 - beq, bne
 - jal
- Designed to be sufficient for:
 - Compilers
 - Operating systems

R-format

31	25 24	20 19	15 14	12 11	7 6	0
funct7	rs2	rs1	funct3	rd	opcode	
7	5	5	3	5	7	
0000000	src2	src1	ADD/SLT/SLTU	dest	OP	
0000000	src2	src1	AND/OR/XOR	dest	OP	
0000000	src2	src1	SLL/SRL	dest	OP	
0100000	src2	src1	SUB/SRA	dest	OP	

ADD rd, rs1, rs2

ADD x9, x21, x9

Rd <--- [rs1] + [rs2]

Why are rs1, rs2 and
rd 5 bit wide

Others:

- Arithmetic: ADD, SUB
- Compare: signed, unsigned--Set if Less Than
- Logical: AND, OR, XOR
- Shift: Left, Right-Logical, Right-Arithmetic

Register Fields in Instructions

- **rd**: destination register
- **rs1**: source register 1
- **rs2**: source register 2
- **imm**: immediate constant encoded in instruction

General Rule

$$\text{Memory}[rs1 + imm] \leftarrow rs2$$

- Address is computed using base register and immediate
- Data comes from register *rs2*

Upper Immediate Instructions

- **LUI (Load Upper Immediate)**

$$rd \leftarrow imm \ll 12$$

- Loads 20-bit immediate into upper bits
- Lower 12 bits are set to zero
- **AUIPC (Add Upper Immediate to PC)**

$$rd \leftarrow PC + (imm \ll 12)$$

- **SLL**: Logical left shift

$$rd \leftarrow rs1 \ll rs2$$

- **SLLI**: Shift using immediate shift amount
- **Arithmetic shifts** preserve sign bit

Compare Instructions

- Perform comparison between registers
- Result stored in *rd* as:
 - 1 if condition is true
 - 0 otherwise
- Example:

$$rd = (rs1 < rs2)?1 : 0$$

Branch Instructions

- Conditional control-flow instructions
- General behavior:

if condition true: $PC \leftarrow PC + imm$

else: $PC \leftarrow PC + 4$

- Examples:
 - BEQ, BNE
 - BLT, BGE

Branch Immediate Encoding

- Branch immediate is a signed 12-bit value
- Represents a **half-word offset**
- Last bit is implicitly zero
- Target address:

$$PC_{target} = PC + (imm \ll 1)$$

- Ensures proper instruction alignment

Why Immediate Bits Are Split

- Immediate bits are split across instruction fields
- Saves encoding space
- Hardware reconstructs the immediate during decode
- Common design choice in RISC architectures

Summary

- RISC-V ISA defines the programmer-visible behavior
- RV32I provides a simple, clean base instruction set
- Load-store architecture simplifies datapath design
- Instruction encoding is optimized for efficiency and extensibility